

Website Integration Plan

1. Frontend (UI for users):

- UI/UX will be based on the final designs produced by my teammates. Can be integrated into Streamlit application, which can then be hosted online using Streamlit Cloud for free or firebase hosting.

2. JWT Authentication

- Can integrate the JWT authentication module into the firebase application by validating Firebase-issued tokens inside the Python backend.

Potential Integration:

1. Firebase handles logins and issues JWT's when a user signs in
2. Firebase returns a JWT token to the client (the Streamlit app)
3. The client sends that token with each request to the Python backend
4. The Python backend uses the "Firebase Admin SDK" to validate the token and confirm the user is valid.

The flow would be: User → Firebase Auth → JWT token → Streamlit frontend → Python backend → Firebase token validation

3. Database (Firebase Realtime Database)

The database that could be used is "Firebase Realtime Database" (Firestore) because it suits the collaborative and dynamic nature of our project. Since our application relies on multiple users being able to contribute stories, illustrations, and interactions in real time, Firestore could help provide us with an easy way to keep everything updated instantly across different pages without having to build and maintain a custom backend. This would effectively integrate with Streamlit and other tools we are already using.

The database would be allocated to store:

- Users (accounts, authentication info, roles like writer/illustrator/admin).
- Logs (activity logs, mood logs, timestamps, severity).
- Stories (chapter text, author name, artwork references).

4. File storage (Cloud Storage for Firebase):

For the websites unstructured binary files we will be using "Cloud Storage for Firebase" to

handle our images, PDF's and videos hosted on the site.

Cloud Storage for Firebase is a good fit because it complements the Realtime Database by handling unstructured files like images, PDFs, and videos, while the database manages structured data such as text and user details. This separation keeps the system efficient and secure, since Firebase integrates file access with authentication. For our storytelling module, it means illustrations can be uploaded, stored safely, and linked back to chapters through simple URLs saved in the database. It's scalable, easy to integrate, and avoids the extra backend complexity.

Example for the storytelling module

1. Illustrator uploads an image (illustration.png).
2. Streamlit app uses Firebase SDK to push it to Cloud Storage.
3. Firebase returns a secure URL.
4. You save that URL in FirebaseDB along with the chapter text.
5. On the Story page, you load text from FirebaseDB + display the image from its URL.

5. Hosting:

Options:

Streamlit Community Cloud is the simplest option for turning the app into a live, accessible website. Since it directly integrates with GitHub, deployment is quick and requires little technical setup, making it a good fit. It provides a free public URL under the streamlit.app domain, and updates are as simple as pushing changes to the repository. However, because this service is designed primarily for demos and prototypes, there are limitations on resources, scaling, and customization, which might hurt UI/UX integration.

Firebase Hosting provides SSL by default and integrates smoothly with Cloud Run or Cloud Functions for handling dynamic backend processes, giving us a pathway toward a production-grade system. The trade-off is that it requires more configuration, and a more complex deployment pipeline, which may slow down immediate progress but would provide a more flexible infrastructure and allow for more customizable UI/UX integration.

Example Architecture Flow

1. User visits hosted Streamlit app. (Streamlit Cloud or Firebase Hosting)
2. User logs in → Streamlit calls Firebase Auth (email/password or Google login).

3. Firebase returns a JWT token to the frontend.

4. Streamlit sends token with requests → Python backend validates token using Firebase Admin SDK.

5. Validated users can:

- Read/write structured data in Firebase Realtime Database
- Upload images/PDFs to Firebase Cloud Storage → URLs stored in database.
- Auto-generate activity/mood logs stored in database.

6. For immediate functionality, the app can be hosted on Streamlit Cloud. For scalability, and UI/UX customisation, Firebase Hosting with Cloud Run could be selected.